StudyAI

Copyright Registration Supporting Document, Copyright Act, 1957 (India)

Author: Neelkamal Gupta · Date: April 2026

GitHub: github.com/neelkamal1969/quiz-project · Live: quiz-project-five-inky.vercel.app

OWNER MEMBERS DETAILS

NAME	ADDRESS WITH EMAIL AND MOBILE NO.
Neelkamal Gupta	**Address: Shimlapuri, LDH-141003(PB-IN)** **Official Email: neelkamal1969.be22@chitkara.edu.in** **Personal Email: neel76441969@gmail.com** **Mobile No.: +91 7527017902**

Section 0 - Overview and Copyright Eligibility

0.1 Nature of the Work

StudyAI is an original, full-stack software application developed independently by Neelkamal Gupta as an academic and portfolio project. The platform converts static study materials - uploaded photographs or manually entered topics - into dynamic, personalised question-and-answer sets using browser-native Optical Character Recognition (OCR) and a Large Language Model backend powered by Google Gemini.

The project comprises two distinct layers of original source code: a React-based frontend (compiled using Vite) organised across nine component and page files, and a Node.js/Express backend that handles AI prompting, user authentication, data persistence, and administrative analytics. The substantial majority of the code and all architectural and design decisions were made by the author, reflecting independent intellectual judgment throughout.

0.2 Copyright Eligibility Under Indian Law

Under Section 2(o) of the Copyright Act, 1957 (India), computer programmes are treated as literary works and are therefore entitled to copyright protection. Section 2(ffc), as amended, defines a computer programme as “a set of instructions expressed in words, codes, schemes or in any other form, including a machine readable medium, capable of causing a computer to perform a particular task or achieve a particular result.”

This codebase satisfies the originality threshold under Indian copyright law. That threshold is low - requiring only that the work originate from the author and not be copied - and this project clearly meets it. The creative expression is embedded in the selection and arrangement of algorithmic logic, the precise wording of AI prompts, the original component architecture, the custom visualisation code, and the layered security design. The iterative development evidenced by v1, v2, and v3 versions of several files further demonstrates a genuine, progressive authorship process.

0.3 Basis for Grant

The likelihood of successful registration is strong for the following reasons: the work is unambiguously original and created by a single identifiable author; it constitutes an original selection and arrangement of logic with creative expression throughout; and the application is already deployed and publicly accessible, demonstrating its concrete and fixed form. The work is not a mere compilation of library calls but an integrated platform with original prompt engineering, custom algorithmic implementations, and a distinctive UI authored from scratch.

0.4 Technology Stack

Frontend: React 18, Vite, Tailwind CSS, React Router, Tesseract.js (browser-native OCR)

Backend: Node.js, Express, bcryptjs, jsonwebtoken (JWT), Helmet, express-rate-limit, express-validator

Database: Neon PostgreSQL (serverless)

AI Engine: Google Gemini API

Deployment: Vercel

Section 1 - server/index.js

Backend API Server - Express, Gemini AI, JWT Auth, PostgreSQL

This file constitutes the entire backend of the StudyAI platform - a single-file Express server that orchestrates seven functional layers: middleware configuration, rate limiting, Gemini AI integration, user authentication (bcrypt + JWT), profile management, vault persistence, search logging, and an admin analytics endpoint.

The dual rate-limiting strategy applies a general limiter (100 requests per 15 minutes) globally, and a stricter AI limiter (10 requests per 60 seconds) specifically to the two Gemini endpoints. This protects both the server and the upstream API quota simultaneously.

The personalised AI prompting logic is among the most original aspects of the codebase. Each prompt dynamically incorporates the user's age, gender, academic degree, difficulty preference, and question style to transform a generic LLM into a tailored academic tutor. The precise wording, structure, and fallback values of the prompt templates are entirely the author's own expression.

1.1 - Middleware Stack and Rate Limiters

```
const app = express();

app.use(helmet());

app.use(cors());

app.use(express.json());



const generalLimiter = rateLimit({

  windowMs: 15 * 60 * 1000,

  max: 100,

  message: { error: 'Too many requests. Please try again later.' }

});

app.use(generalLimiter);
```

```
const aiLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 10,
  message: { error: 'Too many AI requests. Please wait 60 seconds.' }
});
```

1.2 - parseGeminiJSON (Defensive LLM Output Parser)

```
const parseGeminiJSON = (text) => {
  try {
    let clean = text.replace(/```json|```/g, '').trim();
    const match = clean.match(/\[[\s\S]*\]/);
    if (match) clean = match[0];
    const parsed = JSON.parse(clean);
    return Array.isArray(parsed) ? parsed : [parsed];
  } catch (err) {
    throw new Error('AI did not return valid JSON');
  }
};
```

1.3 - POST /page: Personalised OCR-to-Questions Endpoint (excerpt)

```
const prompt = `
  Act as a highly specialised Personal Academic AI Tutor.
  USER CONTEXT:
  - Age: ${profile.age || 'Not specified'}
  - Academic Degree: ${profile.degree || 'General Education'}
  - Knowledge Level: ${profile.difficulty || 'Intermediate'}
  - Question Style: ${profile.question_level || 'Conceptual'}
  TASK: Analyse the source material and generate all possible
  high-quality Q&A pairs tailored to this user profile.
  OUTPUT: Return ONLY a valid JSON array.
  Format: [{ "question": "...", "answer": "..." }]
`;
```

Section 2 - AdminLogs.jsx

Admin Analytics Dashboard - Custom SVG Charts, Real-time Polling

AdminLogs.jsx is a self-contained administrative dashboard. All chart rendering, analytics computation, and data visualisation are written from scratch without any charting library, using React hooks and SVG.

The computeAnalytics function performs five simultaneous aggregations over the raw log data in a single O(n) pass: topic frequency, user frequency, hourly distribution (24 buckets), today's count, and top-N rankings. The HourlyChart renders 24 SVG bars with the current hour dynamically highlighted.

The WordCloud scales font size and opacity per-topic based on relative frequency. Auto-refresh polls every 30 seconds via setInterval with cleanup on unmount.

2.1 - Single-Pass Analytics Algorithm

```
const computeAnalytics = (logs) => {

  const topicCount = {};

  const userCount  = {};

  const hourBucket = Array(24).fill(0);

  const today      = new Date().toDateString();

  let   todayCount = 0;



  logs.forEach((l) => {

    const t = (l.topic || '').trim().toLowerCase();

    const e = (l.email || '').trim().toLowerCase();

    topicCount[t] = (topicCount[t] || 0) + 1;

    userCount[e]  = (userCount[e]  || 0) + 1;

    const h = new Date(l.search_time).getHours();

    hourBucket[h]++;

    if (new Date(l.search_time).toDateString() === today) todayCount++;

  });

  // ... returns topTopics, topUsers, hourBucket, totals

};
```

Section 3 - PhotoToQuestions.jsx

Image OCR Pipeline - Bloom's Taxonomy, Flesch-Kincaid, Practice Mode

PhotoToQuestions.jsx implements a multi-stage pipeline: drag-and-drop image upload, browser-native OCR via Tesseract.js with real-time progress reporting, backend AI question generation, local analytics, and an interactive practice mode - all without any external UI component library.

The Bloom's Taxonomy classifier (classifyQuestion) matches each question against a curated keyword list for all six cognitive levels, annotating questions with their level badge in real time. The Flesch-Kincaid readability scorer implements the full reading ease formula from scratch including a heuristic syllable counter (cntSyl). The custom keyword extractor applies a stop-word filter and term-frequency ranking to produce a live word cloud from the OCR text.

3.1 - Bloom's Taxonomy Classifier

```
const BLOOMS = [

  { level: 'Remember',   keywords: ['define','list','recall','name','identify']
},

  { level: 'Understand', keywords: ['explain','summarize','classify','compare']
},

  { level: 'Apply',      keywords: ['use','solve','demonstrate','calculate']   },

  { level: 'Analyze',    keywords: ['analyze','differentiate','examine']       },
```

```
  { level: 'Evaluate',   keywords: ['evaluate','judge','justify','critique']   },
  { level: 'Create',     keywords: ['create','design','compose','construct']   },
];


function classifyQuestion(q) {
  const lower = q.toLowerCase();
  for (const b of BLOOMS)
    if (b.keywords.some(k => lower.includes(k))) return b;
  return BLOOMS[1];
}
```

3.2 - Flesch-Kincaid Readability Scorer (Original Implementation)

```
function fleschKincaid(text) {
  const sentences = Math.max(1,
text.split(/[.!?]+/).filter(s=>s.trim().length>3).length);
  const words = text.trim().split(/\s+/).filter(Boolean);
  const wc = Math.max(1, words.length);
  const syllables = words.reduce((a, w) => a + cntSyl(w), 0);
  const score = Math.max(0, Math.min(100,
    206.835 - 1.015*(wc/sentences) - 84.6*(syllables/wc)));
  // maps to label/color: Easy, Moderate, Difficult, Very Hard
  return { score: Math.round(score), wordCount: wc, sentenceCount: sentences };
}
```

Section 4 - ResultSection.jsx

3D Flip Flashcard System - Swipe, Speech, Guest-Aware Save

ResultSection.jsx renders AI-generated Q&A pairs as interactive 3D flip cards, used by both the PhotoToQuestions and ValueToQuestions pages. The 3D animation uses CSS preserve-3d and rotateY transforms with a split-key rendering strategy: a slideKey triggers re-mount for the slide-in animation, while the flip div is unkeyed so its CSS transform persists correctly across navigations.

The guest-aware save architecture derives an isGuest flag from localStorage at render time (outside any hook). When a guest attempts to save, a GuestSaveBanner is shown instead of an error alert, auto-dismissing after 5 seconds. Touch gesture support computes swipe distance from onTouchStart/onTouchEnd handlers: swipes over 52px navigate cards; shorter gestures flip the card. Keyboard navigation attaches to window.keydown with guards against input focus conflicts.

4.1 - Guest-Aware Save and Touch Gesture Logic

```
const loggedInUser = getLoggedInUser();   // safe localStorage read
const isGuest      = !loggedInUser;


const handleSaveClick = (e, item, index) => {
```

```
  e.stopPropagation();
  if (isGuest) { setShowGuestBanner(true); return; }
  isSaved ? handleUnsave(e, index) : handleSave(e, item, index);
};


const onTouchEnd = (e) => {
  const dx = e.changedTouches[0].clientX - touchStartX.current;
  if (Math.abs(dx) > 52) dx < 0 ? navigate('next') : navigate('prev');
  else setIsFlipped(f => !f);
  touchStartX.current = null;
};
```

Section 5 - ValueToQuestions.jsx

Topic-Based Quiz Generator - Guest-Safe Payload, Fire-and-Forget Logging

ValueToQuestions.jsx implements topic-based quiz generation for both authenticated users and guests. The key architectural decision is the guest-safe payload construction: the express-validator body('userId').optional().isInt() rule is marked optional, meaning omitting the field entirely passes validation and the backend falls back to an empty profile. The handleSearch function omits userId from the request body when no user is in localStorage, rather than sending an invalid value.

Search logging runs as a concurrent fire-and-forget fetch call using .catch(()=>{}) to suppress errors, so logging never delays the user's results. URL encoding via encodeURIComponent prevents injection through topic names containing special characters.

5.1 - Guest-Safe Payload Construction

```
const payload = { count: Number(count) };
if (userData.id) payload.userId = userData.id;
// userId is omitted for guests - backend validator is .optional()
// so absent field passes validation and falls back to empty profile


// Fire-and-forget logging - never blocks the main response
if (userData.email) {
  fetch(`${baseUrl}/log-search`, {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: userData.email, topic }),
  }).catch(() => {});
}
```

Section 6 - LoginPage.jsx & SignUpPage.jsx

Authentication UI - Inline Validation, Dark Glassmorphism Design

LoginPage.jsx implements user login with a dark background, layered radial-gradient orbs, and a frosted-glass card - all in inline styles with no external UI library. Email validation uses an RFC 5322-inspired regex triggered on blur and thereafter on-change, avoiding intrusive immediate feedback. The redirect-after-login decision is made client-side: if the returned user object has no degree field, the user is routed to profile setup; otherwise to the main page.

SignUpPage.jsx mirrors the visual design of LoginPage.jsx and extends form logic to three simultaneously validated fields. All three validators run before the network call so all field errors appear at once. Navigation after signup passes route state ({ signedUp: true }) so the login page can surface a success message.

6.1 - RFC 5322-Inspired Email Validation and Post-Login Redirect

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;



// After successful login:

if (!data.user.degree) {

  navigate('/profile-setup');

} else {

  navigate('/imageInput');

}
```

Section 7 - ProfileSetup.jsx

Multi-Step Profile Wizard - Constellation Canvas, Degree Autocomplete

ProfileSetup.jsx implements a three-step onboarding wizard. The ConstellationBg component is an animated HTML Canvas particle graph: nodes move with random velocities, bounce off edges, and are connected with semi-transparent lines when within 140 pixels. Each node is rendered as a radial gradient glow. This is entirely original Canvas animation code.

The DegreeInput component implements a searchable autocomplete dropdown filtered from 33 degree names using case-insensitive substring matching. It uses onMouseDown (not onClick) to prevent the blur event from closing the dropdown before the selection fires. Multi-step navigation calls renderStep1/2/3 as plain functions (not JSX components) to avoid reconciliation re-mounts between steps.

7.1 - ConstellationBg Canvas Particle Animation (excerpt)

```
const draw = () => {

  ctx.clearRect(0, 0, W, H);

  nodes.forEach(n => { n.x += n.vx; n.y += n.vy;

    if (n.x < 0 || n.x > W) n.vx *= -1;

    if (n.y < 0 || n.y > H) n.vy *= -1;

  });

  for (let i = 0; i < nodes.length; i++)
```

```
    for (let j = i+1; j < nodes.length; j++) {
      const dist = Math.hypot(nodes[i].x-nodes[j].x, nodes[i].y-nodes[j].y);
      if (dist < 140) {
        ctx.strokeStyle = `rgba(79,70,229,${0.11*(1-dist/140)})`;
        ctx.beginPath(); ctx.moveTo(nodes[i].x,nodes[i].y);
        ctx.lineTo(nodes[j].x,nodes[j].y); ctx.stroke();
      }
    }
  animRef.current = requestAnimationFrame(draw);
};
```

Section 8 - Header.jsx

Repositionable Navigation Bar - 3D Tilt, Dock Picker, Mobile Menu

Header.jsx implements a navigation bar that can be repositioned to three dock positions (top, left, right) at runtime. A dynamically injected style tag adjusts body padding to compensate for the nav's physical position. In top-dock mode, a 3D tilt effect responds to cursor position by translating mouse displacement into rotateX/rotateY transforms (±4 degrees). In side-dock modes, the tilt is suppressed. The dock picker closes on outside-click via a document mousedown listener with useEffect cleanup.

8.1 - 3D Tilt on Mouse Move

```
const handleMouseMove = (e) => {
  if (dock !== 'top') return;
  const rect = navRef.current?.getBoundingClientRect();
  if (!rect) return;
  const dx = (e.clientX - (rect.left+rect.width/2))  / (rect.width/2);
  const dy = (e.clientY - (rect.top+rect.height/2)) / (rect.height/2);
  setTilt({ x: dy * -4, y: dx * 4 });
};


const tiltStyle = dock === 'top'
  ? { transform: `perspective(600px) rotateX(${tilt.x}deg) rotateY(${tilt.y}deg)`
}
  : {};
```

Section 9 - HomePage.jsx & Vault.jsx

Dashboard - Auth-Conditional Rendering, SVG Activity Chart, Export

HomePage.jsx serves as both a public landing page and a personalised dashboard. It detects authentication state from localStorage on mount and conditionally renders entirely different content. For authenticated users it fetches vault data, displays preview cards and statistics, and renders an SVG-based activity trend bar chart. Both CSV and PDF export are implemented client-side without any library: CSV uses Blob and URL.createObjectURL; PDF opens a new window, writes formatted HTML, and calls window.print() after a 300ms delay.

Vault.jsx implements the saved flashcard library with search filtering, multi-mode sorting, a detail modal, and a 3D flip practice mode. The getTagCloud function extracts keyword frequencies from card text, filters a stop-word list, and dynamically scales font size proportionally to frequency. The filteredCards derived value chains .filter() and .sort() for live search with three sort modes.

9.1 - Vault Tag Cloud and Filtered Cards

```
const getTagCloud = () => {

  const text = cards.map(c => c.question.toLowerCase()).join(' ');

  const words = text.split(/\s+/).filter(w =>

    w.length > 3 && !['the','and','for','you','this','that'].includes(w)

  );

  const freq = {};

  words.forEach(w => { freq[w] = (freq[w] || 0) + 1; });

  return Object.entries(freq)

    .sort((a,b) => b[1]-a[1]).slice(0,12)

    .map(([word,count]) => ({ word, size: Math.max(14, 14+count*6) }));

};
```

Declaration of Authorship

I, Neelkamal Gupta, hereby declare that all source code excerpts reproduced in this document are my own original work. The code was written independently over the course of the development of the StudyAI platform. No portion of this code has been copied from any existing published software or codebase. All third-party libraries used - Express, React, Tesseract.js, Google Generative AI SDK, bcryptjs, jsonwebtoken, Helmet, express-rate-limit, express-validator, and Neon serverless PostgreSQL - are used via their public APIs in accordance with their respective open-source licences, and all integration code connecting those libraries is entirely my own original work.

The version history preserved in the commented sections of several files (AdminLogs.jsx v1/v2/v3, Header.jsx v1/v2, ResultSection.jsx v1/v2/v3, and others) provides direct evidence of the iterative development process and the progressive design decisions made by the author. Each version reflects genuine intellectual progression: from basic functionality to production-ready implementations with improved error handling, performance optimisation, and feature depth.

This document was prepared in support of a copyright registration application filed under the Copyright Act, 1957 (India). The work is an original literary work in the form of a computer programme as defined under Section 2(ffc) of the Act. All rights are reserved by the author.

Neelkamal Gupta

GitHub: github.com/neelkamal1969

Live Application: quiz-project-five-inky.vercel.app

Repository: github.com/neelkamal1969/quiz-project

Date: April 2026